

Merge Sort

Divide and Conquer: A problem is divided into multiple sub-problems. Each sub-problem is solved individually. Finally, sub-problems are combined to form the final solution.

Divide: this process divides the problem into smaller sub-problems.

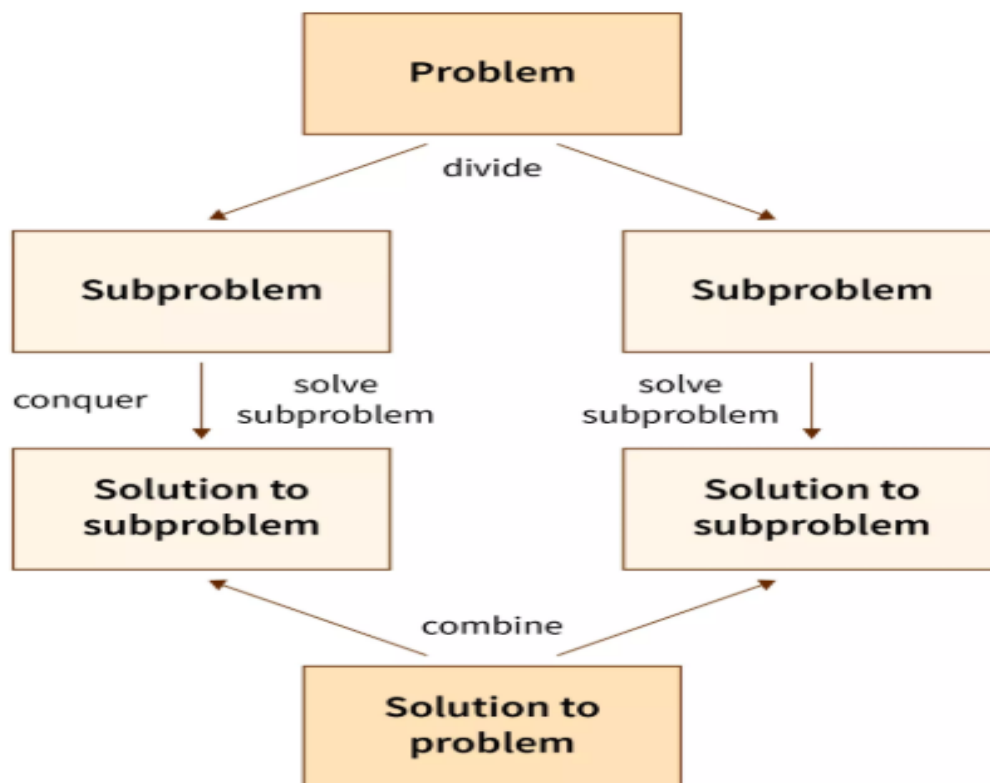
If **mid** is the half-way point between **lb** and **ub**, then we can split the subarray **a[lb...ub]** into two arrays **a[lb,mid]** and **a[mid+1, ub]**.

Conquer: the sub-problems are solved recursively.

In the conquer step, sort both the subarrays **a[lb..mid]** and **a[mid+1, ub]**. If the base case is not yet reached, divide both these subarrays and try to sort them.

Combine: the solutions of the sub-problems are combined in order to achieve the final solution.

When the conquer step reaches the base step and we get two sorted subarrays **a[lb...mid]** and **a[mid+1, ub]** for array **a[lb..ub]**, we combine the results by creating a sorted array **a[lb..ub]** from two sorted subarrays **a[lb..mid]** and **a[mid+1, ub]**.



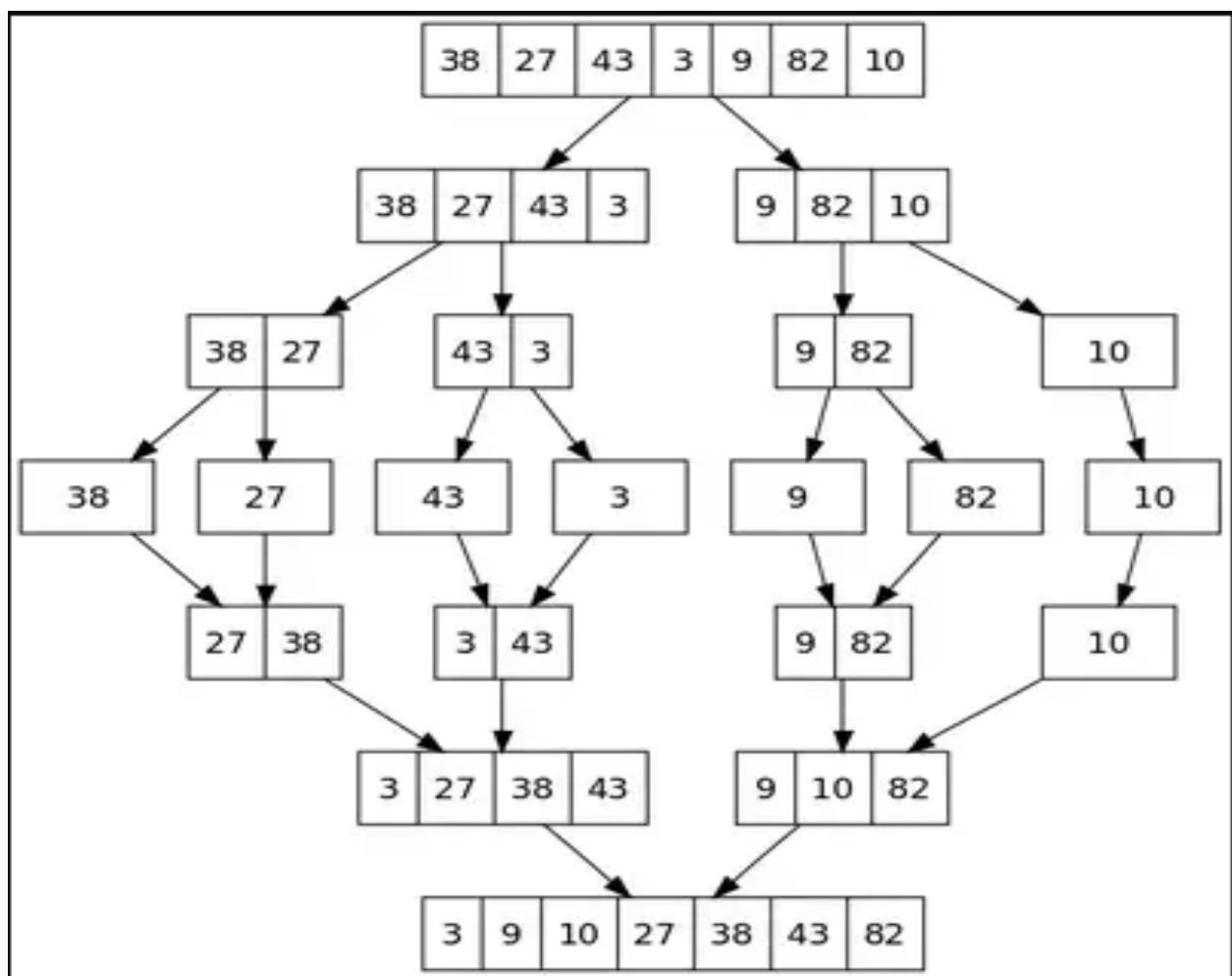
MergeSort Algorithm

Merge sort is the sorting techniques that follows the divide and conquer approach.

It divides the list into sub-lists. The sub-lists are divided again and again into halves until the list cannot be divided further. Then combine the pair of one element lists into two-element lists, sorting them in the process. The sorted two-element pairs are merged into the four-element lists, and so on until we get the sorted list.

How it works:

1. Divide the unsorted array into two sub-arrays, half the size of the original.
2. Continue to divide the sub-arrays as long as the current piece of the array has more than one element.
3. Merge two sub-arrays together by always putting the lowest value first.
4. Keep merging until there are no sub-arrays left.



Implementation:

The **MergeSort()** function repeatedly divides the given array into two parts until we reach a stage where we try to perform MergeSort on a subarray of size 1 i.e. $lb == ub$.

After that, combines the sorted arrays into larger arrays until the whole array is merged.

Merge Sort Complexity:

Time Complexity	
Best Case	$O(n \cdot \log n)$
Average Case	$O(n \cdot \log n)$
Worst Case	$O(n \cdot \log n)$
Space Complexity	$O(n)$

- **Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of merge sort is **$O(n \cdot \log n)$** .
- **Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of merge sort is **$O(n \cdot \log n)$** .
- **Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of merge sort is **$O(n \cdot \log n)$** .
- The **space complexity** of merge sort is $O(n)$. It is because, in merge sort, an extra variable is required for swapping.

Advantages:

1. More efficient and works faster than quick sort in case of larger array size.

Disadvantages:

1. Not in Place because it requires additional memory space to store auxiliary arrays.

The important part of the merge sort is the **MERGE()**. This function performs the merging of two sorted sub-arrays that are **$a[lb \dots mid]$** and **$a[mid+1 \dots ub]$** , to build one sorted array **$a[lb \dots ub]$** . So, the inputs of the **MERGE** function are **$a[]$, lb , mid , and ub** .

MergeSort in Python

```
def mergeSort(array):
    if len(array) > 1:
        # mid is the point where the array is divided into two subarrays
        mid = len(array)//2
        L = array[:mid]
        R = array[mid:]

        mergeSort(L)
        mergeSort(R)

        i = j = k = 0

        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                array[k] = L[i]
                i = i + 1
            else:
                array[k] = R[j]
                j = j + 1
            k = k + 1

        while i < len(L):
            array[k] = L[i]
            i = i + 1
            k = k + 1

        while j < len(R):
            array[k] = R[j]
            j = j + 1
            k = k + 1

array = [6, 5, 12, 4, 10, 9, 1]
mergeSort(array)
print("Sorted array is: ", array)
```